
Table of Contents

PREAMBLE	1
QUESTION 1: COMMENTING	1
QUESTION 2	4
2(a) PLOT FIRST FOUR PERIODS	4
2(b) CREATE AND SUBMIT .WAV FILE	6
2(b) CREATE AND SUBMIT .WAV FILE	6
2(c) PLOT FIRST FOUR PERIODS	6
2(d) PLOT FIRST FOUR PERIODS	6
QUESTION 3	6
3(a) CREATE SOUND	7
3(b) MODIFY FUNCTION (key_to_note_trumpet function is at end of file)	7
3(c) CREATE SOUND (build_song_trumpet function is at end of file)	7
3(d) CREATE SOUND (build_song_time function is at end of file)	7
3(e) PLOT COMPARISONS	7
3(f) ANSWER QUESTION	7

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear;  
close all;
```

QUESTION 1: COMMENTING

=====

```
type('eel3135_lab_FourierSynthesis_comment.m')
```

```
%% ACKNOWLEDGEMENTS / REFERENCES:
```

```
% This code uses functions written by Ken Schutte in 2019, which is used to  
% read and decode midi files. The code is under a GNU General  
% Public License, enabling us to run, study, share, and modify the  
% software.
```

```
%
```

```
% More info can be found at: http://www.kenschutte.com/midi
```

```
%% INITIAL SETUP
```

```
clear
```

```
close all
```

```
clc
```

```
%% DEFINE MUSIC
```

```
% INITIAL VARIABLES
```

```
Fs = 44100; % Sampling frequency (Hz)
```

```

% Reads our audio file, finding the pitch, endtime shows duration.
[Notes, endtime] = midiInfo(readmidi('gym.mid'), 0, 2);
L = size(Notes,1);           % How many notes in our midi file

% Builds a song from our notes, gives an amplitudes of 1, defines the key
% of our note, the duration, and takes our sampling frequency.
x = build_song(ones(L,1), Notes(:,3), Notes(:,6)-Notes(:,5), Fs);

% Finds the duration of the notes and finds the total number of samples
tot_samples = ceil(sum(Notes(:,6)-Notes(:,5))*Fs);

% Creates a time vector for creating our waveform
t = 0:1/Fs:(tot_samples-1)/Fs;

% Plots our waveform
figure(1);
subplot(211)
plot(t, x);
xlabel('Time [s]')
ylabel('Amplitude')
% Replots with a smaller scope, 'zooming in'
subplot(212)
plot(t, x);
xlabel('Time [s]')
ylabel('Amplitude')
axis([0 0.1 -1 1]) % Restricts our x-axis to the first 0.1 seconds, and y
axis to -1:1

% ==> Describe what the next line does here <==
input('Click any button to play sound')
soundsc(x, Fs);

% =====
% YOU DO *NOT* NEED TO DESCRIBE THESE LINES (your free to figure it out
though)
W = 0.1;           % Window size
tic;
for mm = 1:ceil(tot_samples/Fs/W)
    % PAUSE UNTIL NEXT FRAME
    xlim([(mm-1)*W+[0 W]]); % Set limits of plot
    tm = toc;                % Check current time
    if mm*W < tm, disp(['Warning: Visualization is ' num2str(mm*W-tm) 's
behind']); end
    drawnow; pause(mm*0.1-tm); % Synchronize with clock
end
% =====

%%
% =====

```

```

% SUPPORTING FUNCTIONS FOUND BELOW
% Add comments appropriately below
% =====

function x = key_to_note(A, key, dur, fs)
% key_to_note: Converts piano key to a sinusoidal waveform
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note

% Calculate the number of samples, creates a vector, and converts the
% key to a frequency.
N = floor(dur*fs);
t = (0:(N-1)).'/fs;
freq = (440/32)*2^((key-9)/12);

% ==> Creates a complex sinusoid, taking the real part for our
% waveform audio
x = real(A*exp(1j*2*pi*freq*t));

end

function x = build_song(As, keys, durs, fs)
% build_song: Creates audio from our note inputs
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%
% Creates zeroes for the samples of the audio
x = zeros(ceil(sum(durs)*fs), 1);
for k = 1:length(keys)

% Generates a note waveform, finds start point of the note based
% on sum of other notes prior
note = key_to_note(As(k), keys(k), durs(k), fs);
start_time = sum(durs(1:k-1));

% Creates start time and duration to make sample indices

```

```

        n1          = floor(start_time*fs) + 1;
        n2          = floor(start_time*fs) + floor(durs(k)*fs);
        x(n1:n2)    = x(n1:n2) + note;

    end

end

```

QUESTION 2

=====

2(a) PLOT FIRST FOUR PERIODS

```

fs = 8000;

f1 = 400;
f2 = 400;
f3 = 600;

T1 = 1/f1;
T2 = 1/f2;
T3 = 1/f3;

t1 = 0:1/fs:4*T1;
t2 = 0:1/fs:4*T2;
t3 = 0:1/fs:4*T3;

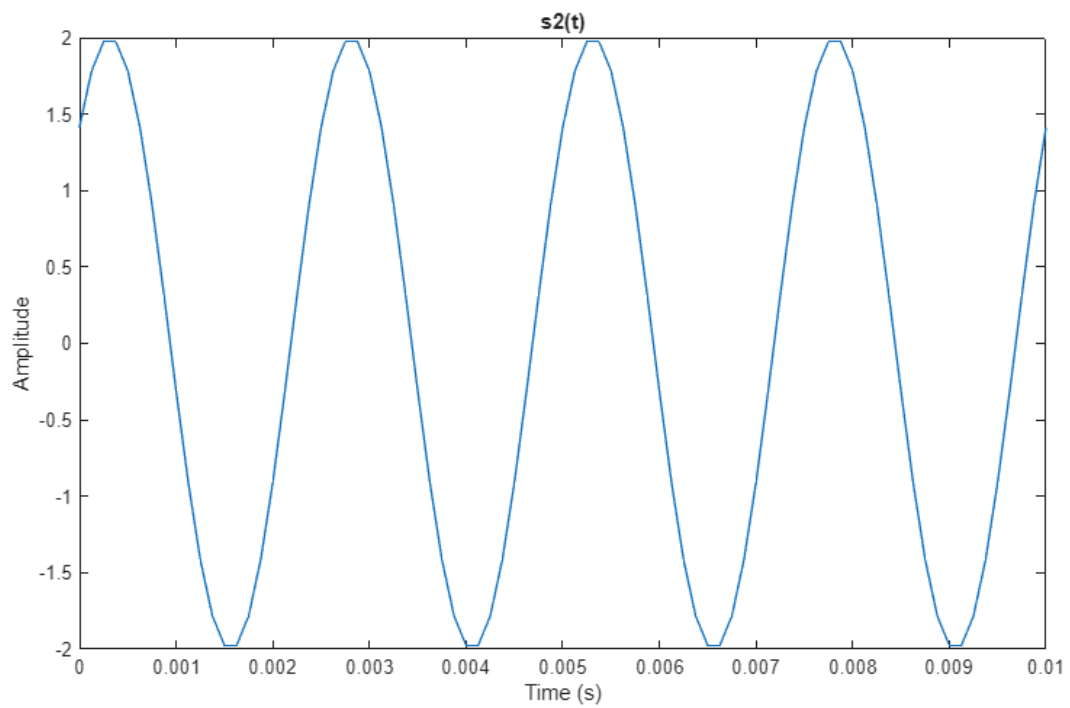
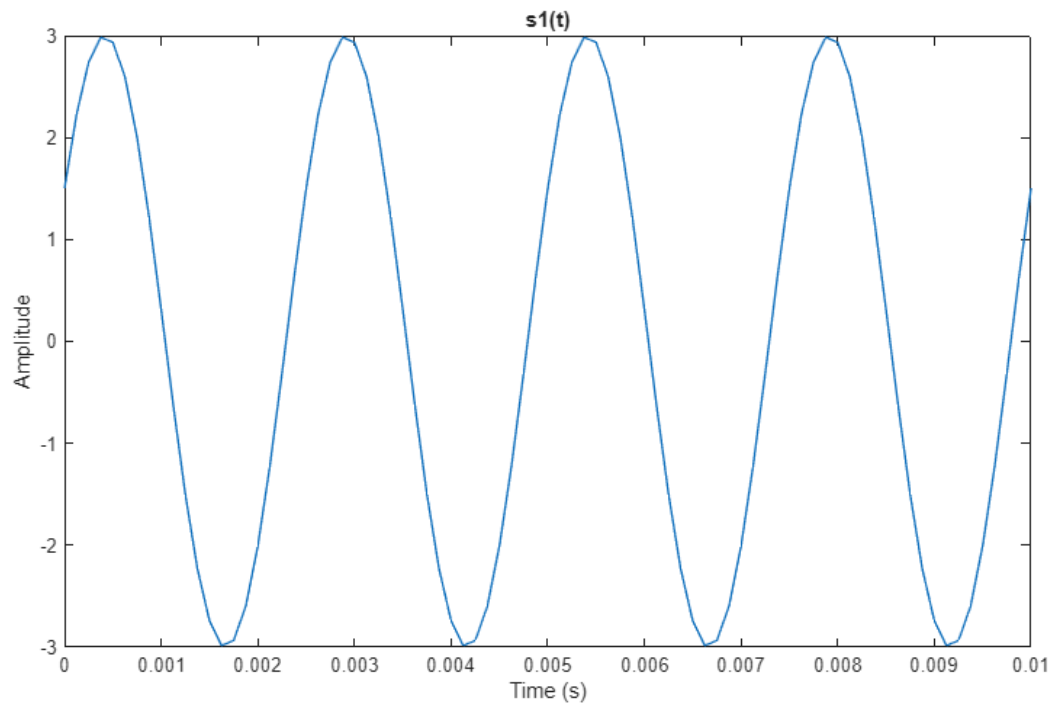
s1 = 3 * cos(800 * pi * t1 - pi/3);
s2 = 2 * cos(800 * pi * t2 - pi/4);
s3 = 2 * cos(1200 * pi * t3 - pi/4);

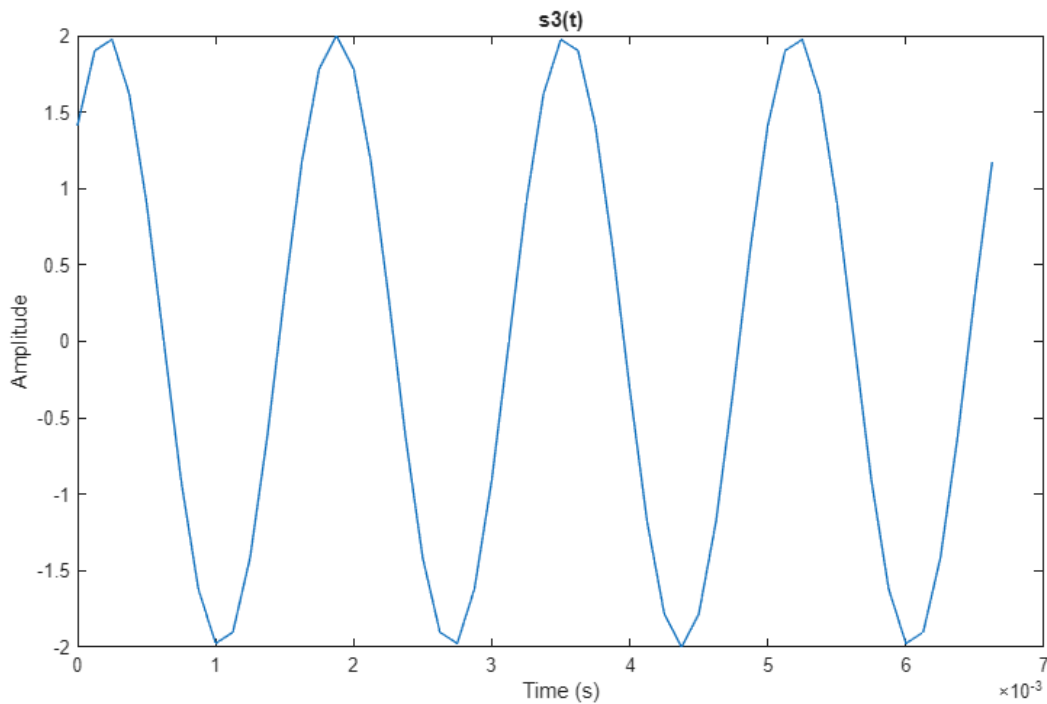
figure;
plot(t1, s1);
xlabel('Time (s)');
ylabel('Amplitude');
title('s1(t)');

figure;
plot(t2, s2);
xlabel('Time (s)');
ylabel('Amplitude');
title('s2(t)');

figure;
plot(t3, s3);
xlabel('Time (s)');
ylabel('Amplitude');
title('s3(t)');

```





2(b) CREATE AND SUBMIT .WAV FILE

2(b) CREATE AND SUBMIT .WAV FILE

```
s1_scaled = s1 / max(abs(s1));  
s2_scaled = s2 / max(abs(s2));  
s3_scaled = s3 / max(abs(s3));
```

```
audiowrite('s1.wav', s1_scaled, fs);  
audiowrite('s2.wav', s2_scaled, fs);  
audiowrite('s3.wav', s3_scaled, fs);
```

2(c) PLOT FIRST FOUR PERIODS

2(d) PLOT FIRST FOUR PERIODS

QUESTION 3

=====

```
fs = 8000;
```

3(a) CREATE SOUND

3(b) MODIFY FUNCTION (key_to_note_trumpet function is at end of file)

Only need to modify function -- this area can be empty

3(c) CREATE SOUND (build_song_trumpet function is at end of file)

3(d) CREATE SOUND (build_song_time function is at end of file)

3(e) PLOT COMPARISONS

3(f) ANSWER QUESTION

===== SUPPORTING FUNCTIONS FOUND BELOW
=====

```
function x = key_to_note(A, key, dur, fs)
% key_to_note:
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note
```

```
    N = floor(dur*fs);
    t = (0:(N-1)).'/fs;
    freq = (440/32)*2^((key-9)/12);
    x = real(A*exp(1j*2*pi*freq*t));
```

```
end
```

```
function x = build_song(As, keys, durs, fs)
% build_song:
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
```

```

%   keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%

x = zeros(floor(sum(durs)*fs), 1);
for k = 1:length(keys)
    note      = key_to_note(As(k), keys(k), durs(k), fs);
    start_time = sum(durs(1:k-1));
    n1        = floor(start_time*fs) + 1;
    n2        = floor(start_time*fs) + floor(durs(k)*fs);
    x(n1:n2)  = x(n1:n2) + note;
end

end

function x = key_to_note_trumpet(A, key, dur, fs)
% key_to_note: Produces a sinusoidal waveform corresponding to a
%   given piano key number
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note

end

function x = build_song_trumpet(As, keys, durs, fs)
% build_song:
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%
```

end

```
function x = build_song_time(As, keys, start_time, end_time, fs)
% build_song:
%
% Input Args:
%     As: A length-N array of complex amplitudes for building notes
%     keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%     start_time: A length-N array of start times (in seconds) for notes
%     end_time: A length-N array of end times (in seconds) for notes
%     fs: A scalar sampling rate value
%
% Output Args:
%     x: A length-(N*fs) length raw audio signal
%
```

end

Published with MATLAB® R2025a